

Uppsala University
Department of Information Technology

Assignment 3

Distributed Infrastructure, Model Serving and CI/CD

Course: Data Engineering II

Arnab Kumar Ghosh

May 8, 2026

Contents

1	Introduction	2
2	Task 1: Single Server Deployment	2
2.1	Overview	2
2.2	Application Architecture	2
2.3	CloudInit Configuration	2
2.4	Starting the Production VM	3
2.5	Questions	3
3	Task 2: Single Server Deployment with Docker Containers	6
3.1	Overview	6
3.2	Docker Compose Architecture	6
3.3	CloudInit Configuration (with Docker)	7
3.4	Scaling the Workers	7
3.5	Questions	7
4	Task 3: Deployment of Multiple Servers Using Ansible	11
4.1	Overview	11
4.2	Infrastructure	11
4.3	CloudInit Configuration	11
4.4	Ansible Playbook	12
4.5	Questions	14
5	Task 4: CI/CD Using Git Hooks	15
5.1	Overview	15
5.2	Architecture	15
5.3	The post-receive Hook	16
5.4	Running the Pipeline	16
5.5	Questions	17
6	Conclusion	21

1 Introduction

This assignment covers the deployment of a distributed machine learning application across a cloud infrastructure. Assignment 3 consists of four progressive tasks. Task 1 starts with a single server deployed to a single virtual machine using CloudInit for contextualization. Task 2 improves this by containerizing services with Docker and Docker Compose. Task 3 adds multi-server orchestration using Ansible. Task 4 completes the setup with a continuous deployment pipeline using Git hooks. Each task builds on the previous one, introducing new tools and addressing previous limitations.

2 Task 1: Single Server Deployment

2.1 Overview

Task 1 requires deploying a machine learning model serving application on a single virtual machine using CloudInit for dynamic contextualization. The application consists of three components: a Flask web frontend, a Celery worker, and a RabbitMQ message broker. The model is a Keras/TensorFlow neural network trained on the Pima Indians Diabetes dataset. The deployment must be automated from a client VM using the OpenStack Python API. During the first boot, CloudInit is responsible for all software installation and service startup. This eliminates the need for any manual login to the production server.

2.2 Application Architecture

The production server has three components:

- **Flask app** (`app.py`): runs the web frontend on port 5100. It receives prediction requests from the browser and sends them to Celery as tasks.
- **RabbitMQ**: sits between Flask and Celery. It holds tasks in a queue until a worker is ready.
- **Celery worker** (`workerA.py`): takes a task from the queue, loads the Keras model, runs the prediction, and sends the result back.

The data flow is:

Browser → Flask (`app.py`) → RabbitMQ → Celery worker (`workerA.py`) → Flask → Browser

2.3 CloudInit Configuration

The file `cloud-cfg.txt` is given to OpenStack when the production VM is created. CloudInit runs the steps below automatically on the first boot:

Listing 1: `cloud-cfg.txt` – single server without Docker

```
1 #cloud-config
2
3 apt_update: true
4 apt_upgrade: true
5 packages:
6 - python3-pip
7 - python3-dev
8 - build-essential
9 - rabbitmq-server
10
11 byobu_default: system
```

```
12
13 runcmd:
14 - pip3 install "celery" "tensorflow==2.10.0" "amqp" "flask==2.3.1" "future" "numpy
    <2.0"
15 - git clone https://github.com/sztoor/model_serving.git
16 - celery --workdir=/model_serving/single_server_without_docker/production_server \
17     -A workerA worker --detach --loglevel=debug --concurrency=1 -n wkr1@backend
18 - python3 /model_serving/single_server_without_docker/production_server/app.py \
19     --host=0.0.0.0 --port=5100 &
```

2.4 Starting the Production VM

On the client VM, I sourced the OpenStack RC file and installed the OpenStack Python libraries. Then I ran the following command:

Listing 2: Starting the production VM from the client

```
python3 start_instance.py
```

The script read credentials from the environment, created the VM with the CloudInit user-data, and waited until the instance status was ACTIVE. After I attached a floating IP, the application was reachable at:

- Welcome page: <http://130.238.27.96:5100>
- Predictions page: <http://130.238.27.96:5100/predictions>

2.5 Questions

1. Explain how the application works.

The application has three tiers. A client sends input values to the Flask web page. Flask puts the prediction request into a RabbitMQ queue as a Celery task. A Celery worker picks up the task, loads the model from `model.h5` and `model.json`, runs the prediction, and saves the result. Flask then reads the result and shows it on the `/predictions` page. This design keeps the web server fast because heavy model computation runs in a separate worker process.

2. What are the drawbacks of the contextualization strategy adopted in Task 1?

1. **Slow deployment:** every new VM must download and install all packages from the internet at boot time. This takes 10–15 minutes.
2. **No reproducibility:** the script runs `apt_upgrade: true`, so the latest package versions are installed. A new VM created later may get different package versions and behave differently.
3. **No isolation:** all services share the same OS. A package conflict or a crashed service can bring down the whole server.
4. **No automatic restart:** services are started as background processes. If a process crashes, nothing restarts it.
5. **Hard to scale:** adding a Celery worker requires logging in to the VM and running commands manually. There is no automated way to scale.
6. **No persistence:** if the VM is deleted, all data and configuration are lost. The whole setup must be done again from scratch.

3. How can we reduce the deployment time?

Most of the time is spent downloading and installing packages on each new VM. A pre-built Docker image solves this. All dependencies are installed once and saved to an image registry. When the VM boots, it only pulls the image and starts a container. This takes seconds instead of minutes. Another option is to create a custom OpenStack snapshot with all packages already installed. The VM then boots from that snapshot and skips the `apt install` step entirely.

4. Summary

In this task, I deployed a complete machine learning serving stack to a fresh cloud VM with no manual intervention after the VM was created. The only input I provided was a single `cloud-cfg.txt` file passed to the OpenStack API at launch time.

I sourced the OpenStack RC file on the client VM and ran `start_instance.py`, which called the OpenStack API to create the production VM. CloudInit then executed automatically on the first boot. It updated the system packages and installed Python 3, RabbitMQ, and all required pip packages. It then cloned the application repository from GitHub and started both the Celery worker (as a background process) and the Flask web server on port 5100. The entire process from VM creation to a live service took approximately 12–15 minutes without any further input from me.

I attached a floating IP and verified that the three-component architecture—Flask, RabbitMQ, and Celery—worked correctly from the first boot. I submitted input values on the predictions page. They were queued as a Celery task and processed by the worker using the Keras model. The prediction result then appeared on the `/predictions` page. I did not need to start or configure any service manually after the VM became active.

Despite the successful deployment, I observed several important limitations. Packages are downloaded and installed at every new boot. This makes deployment slow and non-reproducible, since different package versions may be installed on different days. Since services run as raw background processes with no process supervisor, a crash is never recovered automatically. All three components share the same OS environment, meaning a dependency conflict or a bad package upgrade can bring down everything at once. To scale, I would have to log in to the VM and start workers by hand.

These problems motivated me to move to Docker containers in Task 2, which provides isolation, automatic restart, and one-command horizontal scaling.

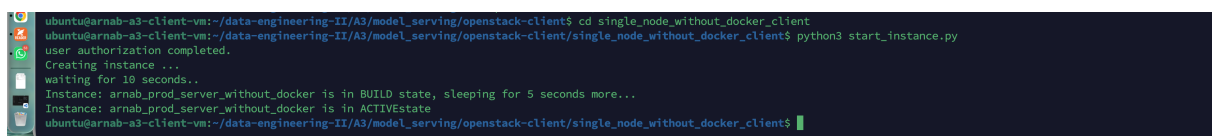
A terminal window screenshot showing the execution of a script. The prompt is `ubuntu@arnab-a3-client-vm:~/data-engineering-II/A3/model_serving/openstack-client$`. The user enters `cd single_node_without_docker_client` and then `python3 start_instance.py`. The output shows "user authorization completed.", "Creating instance ...", "waiting for 10 seconds..", and two status messages: "Instance: arnab_prod_server_without_docker is in BUILD state, sleeping for 5 seconds more..." and "Instance: arnab_prod_server_without_docker is in ACTIVE state". The prompt returns to `ubuntu@arnab-a3-client-vm:~/data-engineering-II/A3/model_serving/openstack-client/single_node_without_docker_client$`.

Figure 1: Client VM running `start_instance.py` to create the production VM via the OpenStack Python API. The script waits until the instance status reaches `ACTIVE` before returning.

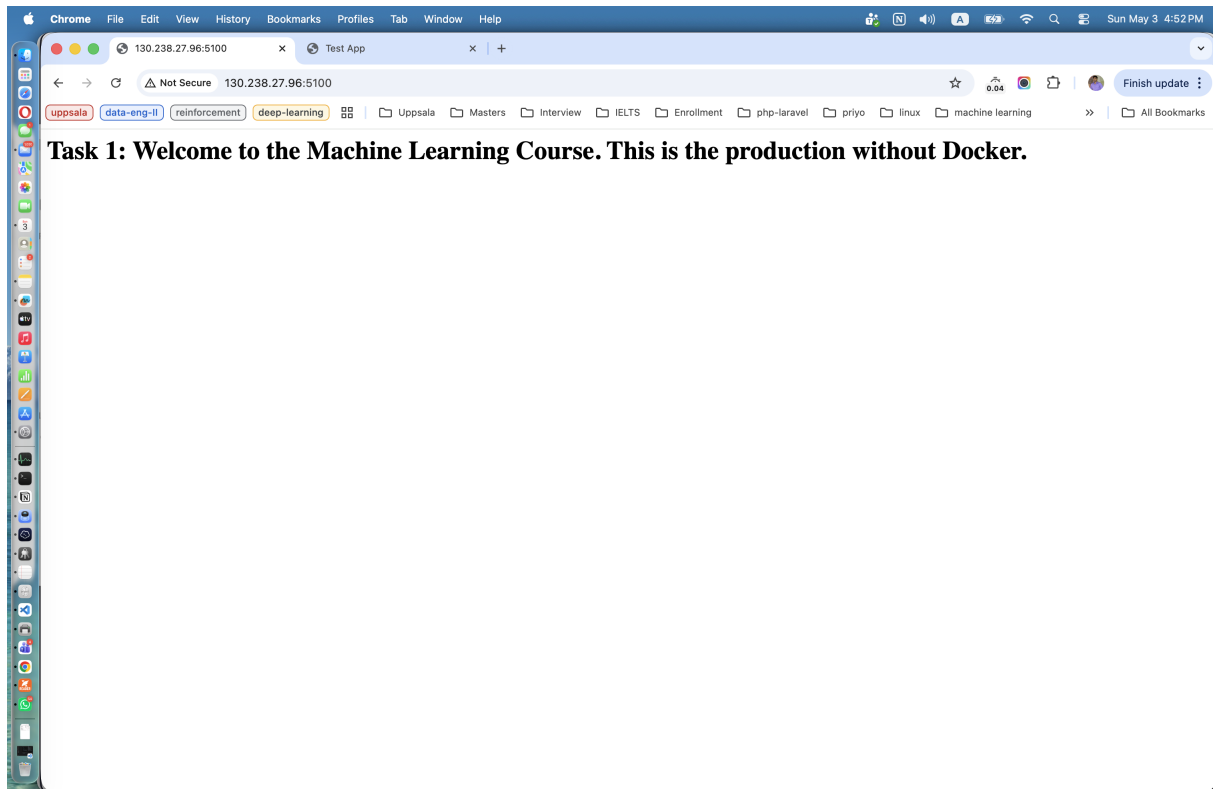


Figure 2: Welcome page of the Flask application served on port 5100 after CloudInit completed all installation and startup steps.

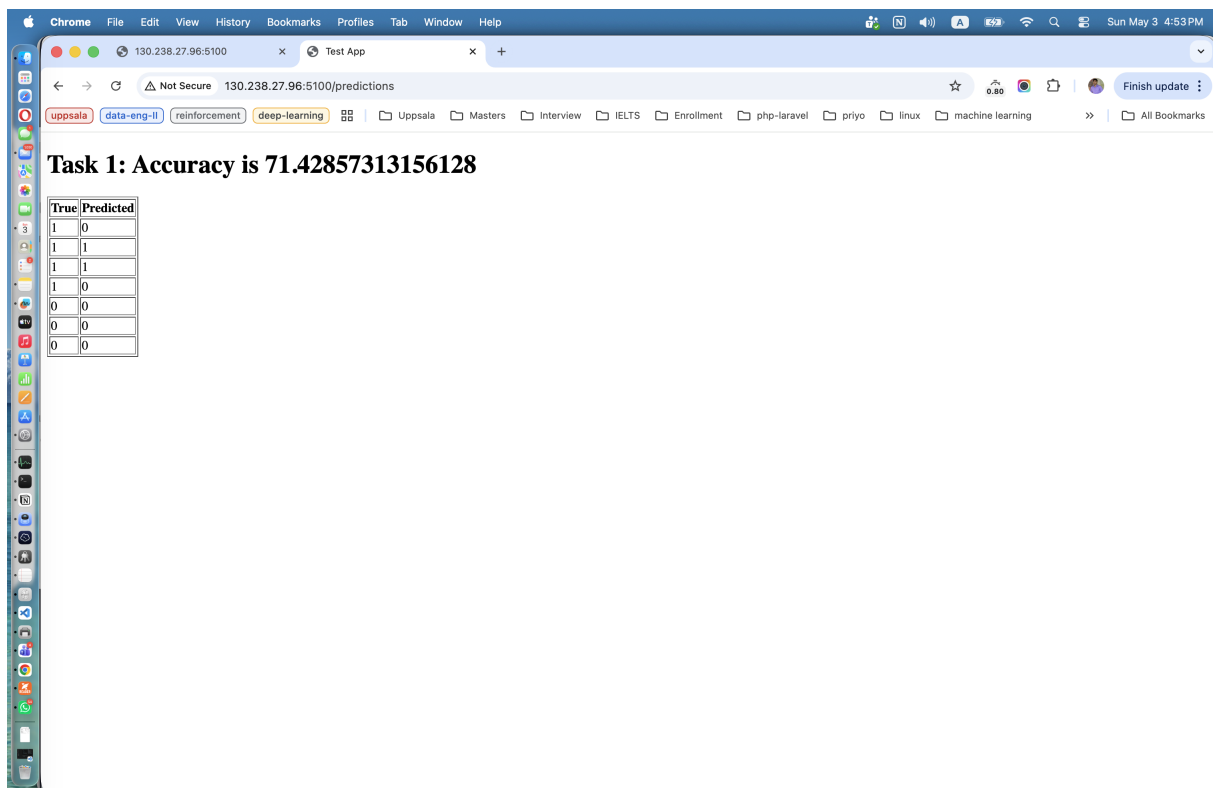


Figure 3: Predictions page showing a diabetes prediction result returned by the Celery worker via RabbitMQ. The result confirms end-to-end task queueing and model inference are working correctly.

3 Task 2: Single Server Deployment with Docker Containers

3.1 Overview

Task 2 requires repeating the deployment from Task 1, but replacing direct package installation with Docker and Docker Compose. Each service must run in its own isolated container built from a Dockerfile. The task also requires demonstrating horizontal scaling by adding and removing Celery worker containers with a single command, without modifying Flask or RabbitMQ.

3.2 Docker Compose Architecture

The `docker-compose.yml` defines three services:

Listing 3: `docker-compose.yml` for the production server

```
1 version: "3"
2 services:
3   web:
4     build:
5       context: .
6     network: host
7     restart: always
8     ports:
9       - "5100:5100"
10    depends_on:
11      - rabbit
12  rabbit:
13    hostname: rabbit
14    image: rabbitmq:3.11-management
15    environment:
16      - RABBITMQ_DEFAULT_USER=rabbitmq
17      - RABBITMQ_DEFAULT_PASS=rabbitmq
18    ports:
19      - "5672:5672"
20      - "15672:15672"
21  worker_1:
22    build:
23      context: .
24    network: host
25    hostname: worker_1
26    entrypoint: celery
27    command: -A workerA worker --loglevel=debug
28    links:
29      - rabbit
30    depends_on:
31      - rabbit
```

- **web**: the Flask frontend, built from the local `Dockerfile`. It listens on port 5100 and waits for RabbitMQ to start first.
- **rabbit**: the official RabbitMQ image with the management plugin. The login credentials are set through environment variables.
- **worker_1**: the Celery worker, built from the same `Dockerfile` but started with the Celery command instead of Flask.

3.3 CloudInit Configuration (with Docker)

The `cloud-cfg.txt` for this task installs Docker CE and starts the application with Docker Compose at boot time. Python packages are no longer installed directly on the VM:

Listing 4: `cloud-cfg.txt` – single server with Docker

```
1 #cloud-config
2 apt_update: true
3 apt_upgrade: true
4 packages:
5   - apt-transport-https
6   - ca-certificates
7   - curl
8   - software-properties-common
9 runcmd:
10  - curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
11  - add-apt-repository -y "deb [arch=amd64] https://download.docker.com/linux/ubuntu
    focal stable"
12  - apt-get update -y
13  - apt-get install -y docker-ce
14  - git clone https://github.com/sztoor/model_serving.git
15  - docker compose -f /model_serving/single_server_with_docker/production_server/docker
    -compose.yml up -d
```

3.4 Scaling the Workers

Horizontal scaling was tested on the production VM after deployment.

Listing 5: Scaling Celery workers from 1 to 3

```
# Initial state -- one worker
sudo docker ps

# Scale up to 3 workers
sudo docker compose up --scale worker_1=3 -d

# Verify three workers are running
sudo docker ps

# Scale back down
sudo docker compose up --scale worker_1=1 -d
```

With three workers, five containers were running at the same time: `rabbit`, `web`, `worker_1_1`, `worker_1_2`, and `worker_1_3`. No changes were needed in Flask or RabbitMQ to add the extra workers.

3.5 Questions

1. What problems Task 2 deployment strategy solves compared to the strategy adopted in Task 1?

Docker fixes several problems from Task 1. Each service runs in its own container, so there are no dependency conflicts between Flask, Celery, and RabbitMQ. The `restart: always` setting means the web container restarts on its own if it crashes. Adding workers takes one command with `-scale`. The `Dockerfile` fixes the exact build environment, so the deployment is more reproducible than installing packages directly with `pip`.

2. What are the outstanding issues that the deployment strategy of Task 2 cannot not address?

- **Single point of failure:** all containers run on one VM. If that VM goes down, the whole application stops.
- **No cross-node scaling:** Docker Compose cannot spread containers across several VMs. All workers share the resources of one machine.
- **Manual deployment:** a new version requires running `docker compose up` by hand. There is no automated pipeline.
- **No load balancing:** the web service has one container. High traffic cannot be spread across multiple web replicas automatically.

3. What are the possible solutions to address those outstanding issues?

A container orchestrator such as **Kubernetes** or **Docker Swarm** can run containers on many VMs, restart failed containers, and deploy updates without downtime. A CI/CD tool such as Git Hooks (Task 4) or GitHub Actions can rebuild and redeploy the application automatically when code changes. A load balancer placed in front of several web containers can spread traffic across all of them.

4. What is the difference between horizontal and vertical scalability? Is the strategy adopted in Task 2 following horizontal or vertical scalability?

Vertical scalability means giving more CPU or RAM to one machine. **Horizontal scalability** means running more instances of a service. Task 2 uses **horizontal** scalability. The `-scale worker_1=3` command adds more worker containers instead of upgrading the VM. Since all containers are still on one host, this is intra-node horizontal scaling. Scaling across multiple hosts would require an orchestrator like Kubernetes.

5. Summary

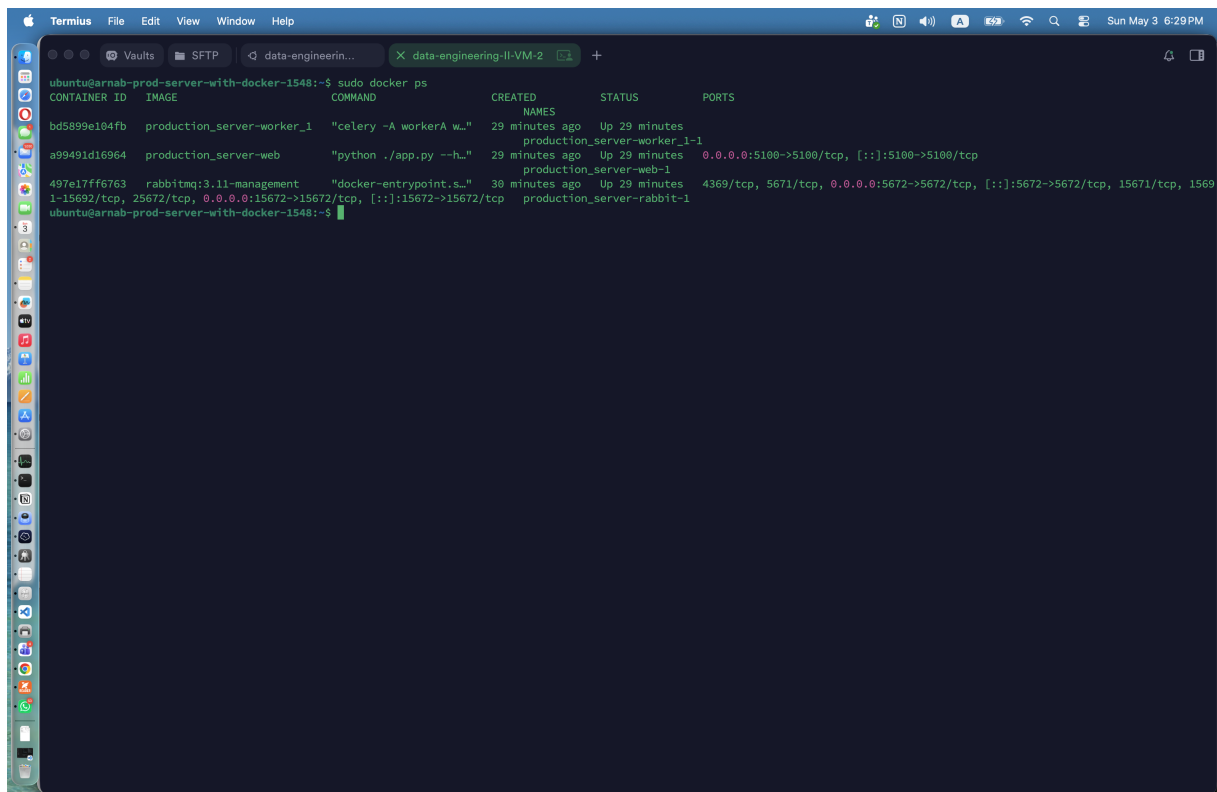
In this task, I containerised every service with Docker and Docker Compose to address the key drawbacks I identified in Task 1. Instead of installing packages directly on the VM, I ran each service—Flask, RabbitMQ, and the Celery worker—in its own isolated container built from a `Dockerfile`. This made the environment fully reproducible: the same image produces identical behaviour regardless of when the VM was created.

I simplified the CloudInit script significantly as a result. I no longer needed it to know anything about Python, pip, or the application itself. I used it only to install Docker CE, clone the repository, and execute `docker compose up -d`. I encapsulated all application complexity inside the `docker-compose.yml` and `Dockerfile`.

I observed two practical gains during this task. First, I benefited from service isolation and automatic restart. The `restart: always` policy on the web container means the web service recovers from crashes without my intervention. The ordered startup via `depends_on` also ensures Flask and the worker always wait for RabbitMQ to be ready. Second, I tested horizontal scaling by running `docker compose up -scale worker_1=3 -d`, which brought the worker count from one to three instantly with no changes required in Flask or RabbitMQ. All three workers shared the same queue and processed tasks in parallel. I confirmed the predictions page worked correctly throughout the scaling test.

The remaining limitation I noted is that all containers still run on a single VM. If that VM fails, the entire application becomes unavailable. Multi-server orchestration with Ansible is introduced in

Task 3 to address this single point of failure.



```
ubunt@arnab-prod-server-with-docker-1548:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
bd5899e194fb	production_server-worker_1	"celery -A workerA w..."	29 minutes ago	Up 29 minutes	
a99491d16964	production_server-web	"python ./app.py --h..."	29 minutes ago	Up 29 minutes	0.0.0.0:5100->5100/tcp, [::]:5100->5100/tcp
497e17ff6763	rabbitmq:3.11-management	"docker-entrypoint.s..."	30 minutes ago	Up 29 minutes	4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp

```
ubunt@arnab-prod-server-with-docker-1548:~$
```

Figure 4: `docker ps` output showing the Flask web service, RabbitMQ, and a single Celery worker running as separate Docker containers before scaling.

```

ubuntu@arnab-prod-server-with-docker-1548:/model_serving/single_server_with_docker/production_server$ sudo docker compose up --scale worker_1=3 -d
WARN[0000] /model_serving/single_server_with_docker/production_server/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Running 5/5
✓ Container production_server-rabbit-1 Running 0.0s
✓ Container production_server-worker_1-1 Running 0.0s
✓ Container production_server-web-1 Running 0.0s
✓ Container production_server-worker_1-3 Started 7.8s
✓ Container production_server-worker_1-2 Started 5.0s
ubuntu@arnab-prod-server-with-docker-1548:/model_serving/single_server_with_docker/production_server$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
73d5c7e29761   production_server-worker_1          "celery -A workerA w..." 13 seconds ago Up 10 seconds
879717f5b42e   production_server-worker_1          "celery -A workerA w..." 13 seconds ago Up 8 seconds
bd5899e104fb   production_server-worker_1          "celery -A workerA w..." 49 minutes ago Up 49 minutes
a99491d16964   production_server-web               "python ./app.py --h..." 49 minutes ago Up 49 minutes    0.0.0.0:5100->5100/tcp, [::]:5100->5100/tcp
497e17ff6763   rabbitmq:3.11-management           "docker-entrypoint.s..." 49 minutes ago Up 49 minutes    4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 1569
1-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp production_server-rabbit-1
ubuntu@arnab-prod-server-with-docker-1548:/model_serving/single_server_with_docker/production_servers$

```

Figure 5: docker ps output after running docker compose up -scale worker_1=3 -d. Three Celery worker containers are now running alongside the web service and RabbitMQ.

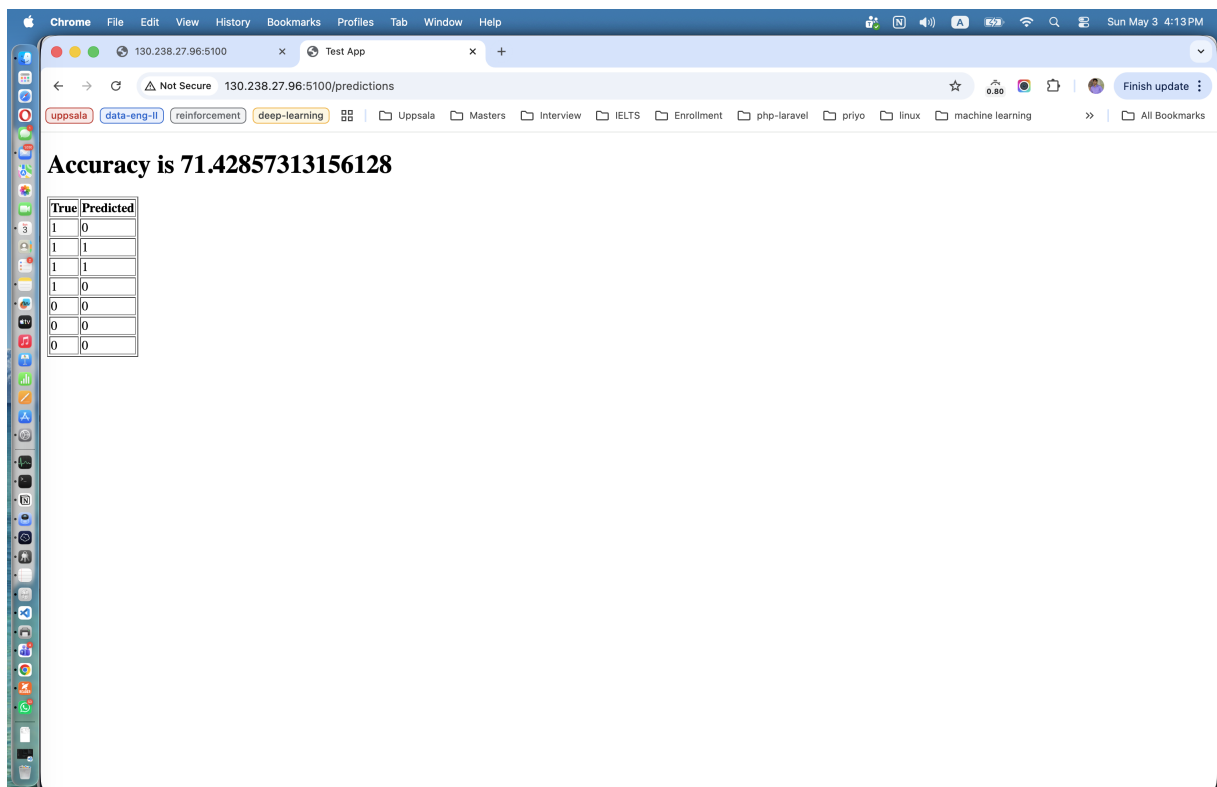


Figure 6: Predictions page confirming the application returns correct results with three concurrent Celery workers processing tasks in parallel.

4 Task 3: Deployment of Multiple Servers Using Ansible

4.1 Overview

Task 3 requires extending the infrastructure to three virtual machines. A client VM must manage both a production server and a development server using the OpenStack Python API to provision them. An Ansible playbook must then configure both servers simultaneously—handling shared setup tasks once and applying server-specific configuration in separate plays. This replaces the per-VM CloudInit approach from Tasks 1 and 2 with a proper configuration management tool that can be re-applied at any time.

4.2 Infrastructure

- **Client VM:** runs the OpenStack client and Ansible. It creates the other two VMs and runs the playbook to configure them.
- **Production server:** runs the Dockerised application from Task 2.
- **Development server:** holds the development code and the model training script.

```

ubuntu@arnab-a3-client-vm:~/data-engineering-II/A3/model_serving/openstack-client/single_node_with_docker_ubuntu_client$ export ANSIBLE_HOST_KEY_CHECKING=False && ansible-playbook configuration.yml --private-key=/home/ubuntu/cluster-keys/cluster-key
[WARNING]: Found both group and host with same name: prodserver
[WARNING]: Found both group and host with same name: devserver

PLAY [all] *****

TASK [Gathering Facts] *****
ok: [prodserver]
ok: [devserver]

TASK [Generate hosts file] *****
changed: [prodserver] => (item=prodserver)
changed: [devserver] => (item=prodserver)
changed: [prodserver] => (item=devserver)
changed: [devserver] => (item=devserver)

TASK [apt update] *****
changed: [prodserver]
changed: [devserver]

TASK [Download git repository] *****
changed: [prodserver]
changed: [devserver]

TASK [Change ownership of a directory] *****
changed: [devserver]
changed: [prodserver]

PLAY [prodserver] *****

TASK [Gathering Facts] *****
ok: [prodserver]

TASK [Extra packages] *****
changed: [prodserver] => (item=apt-transport-https)
ok: [prodserver] => (item=ca-certificates)
ok: [prodserver] => (item=curl)
ok: [prodserver] => (item=software-properties-common)

TASK [apt update] *****
ok: [prodserver]

TASK [Add Docker repository key] *****
changed: [prodserver]

TASK [Add Docker repository] *****
changed: [prodserver]

```

Figure 7: Deployment architecture for Task 3: Client VM managing production and development servers using Ansible.

4.3 CloudInit Configuration

Both VMs use a minimal CloudInit file. Each file creates a single unprivileged user called appuser with passwordless sudo access and the cluster SSH public key. Ansible then connects over SSH and handles all further configuration. The production and development files are structurally identical.

Listing 6: prod-cloud-cfg.txt and dev-cloud-cfg.txt (identical structure)

```

1 #cloud-config
2
3 users:
4   - name: appuser
5     sudo: ALL=(ALL) NOPASSWD:ALL
6     home: /home/appuser
7     shell: /bin/bash
8     ssh_authorized_keys:
9       - ssh-rsa AAAA... # cluster public key (truncated for brevity)
10
11 byobu_default: system

```

4.4 Ansible Playbook

The configuration.yml playbook has three plays. The first play targets all hosts and handles shared setup. The second targets only the production server to install Docker and start the application containers. The third targets only the development server to install the ML packages needed for model training.

Listing 7: configuration.yml

```

1 - hosts: all
2
3   tasks:
4
5     - name: Generate hosts file
6       lineinfile: dest=/etc/hosts
7                   regexp='.*{{ item }}$'
8                   line="{{ hostvars[item].ansible_default_ipv4.address }}"
9                   state=present
10      when: hostvars[item].ansible_default_ipv4.address is defined
11      with_items: "{{ groups['all'] }}"
12      become: true
13
14      #- name: Set hostname
15      #  hostname: name="{{ inventory_hostname }}"
16      #  become: true
17
18      - name: apt update
19        apt: update_cache=yes upgrade=dist
20        become: true
21
22      - name: Download git repository
23        become: true
24        git:
25          repo: 'https://github.com/sztoor/model_serving.git'
26          dest: /model_serving
27
28      - name: Copy docker-compose override to pin RabbitMQ version (avoids 4.x
29        transient_nonexcl_queues error)
30        become: true
31        copy:
32          dest: /model_serving/ci_cd/production_server/docker-compose.override.yml
33          content: |
34            version: "3"
35            services:
36              rabbit:
37                image: rabbitmq:3.12-management

```

```
37
38   - name: Change ownership of a directory
39     become: true
40     file:
41       path: /technical-training
42       state: directory
43       recurse: yes
44       owner: appuser
45       group: appuser
46
47 - hosts: prodserver
48
49 vars_files:
50   - setup_var.yml
51
52 tasks:
53   - name: Extra packages
54     apt: pkg={{item}} state=latest update_cache=true
55     with_items:
56       - apt-transport-https
57       - ca-certificates
58       - curl
59       - software-properties-common
60     become: true
61
62   - name: apt update
63     apt: update_cache=yes upgrade=dist
64     become: true
65
66   - name: Add Docker repository key
67     apt_key:
68       url: https://download.docker.com/linux/ubuntu/gpg
69       state: present
70     become: true
71
72   - name: Add Docker repository
73     command: add-apt-repository -y "deb [arch=amd64] https://download.docker.com/
74 linux/ubuntu focal stable"
75     become: true
76
77   - name: apt update
78     apt: update_cache=yes upgrade=dist
79     become: true
80
81   # Workaround pip issue with MTU mismatch, see: https://github.com/docker/docker.
82   # github.io/pull/9249/files
83   # MTU for SNIC is 1450, Docker default is 1500. Override so that Docker matches the
84   # host.
85   # We create the config file before installing, to avoid any need to restart the
86   # docker daemon.
87   - name: Create Ansible docker config directory.
88     become: true
89     file:
90       path: "/etc/docker"
91       state: directory
92
93   - name: Create Ansible docker file to configure Docker MTU to match SNIC for pip
94     issue
```

```

90     become: true
91     copy:
92         dest: "/etc/docker/daemon.json"
93         content: |
94             {
95                 "mtu": 1450
96             }
97
98     - name: Install Docker
99       apt: pkg=docker-ce state=present update_cache=true allow_unauthenticated=yes
100       become: true
101
102     - name: Running containers
103       become: true
104       shell: docker compose up -d
105       args:
106         chdir: /model_serving/ci_cd/production_server
107
108 - hosts: devserver
109
110 vars_files:
111   - setup_var.yml
112
113 tasks:
114   - name: Extra packages
115     apt: pkg=python3-pip state=latest update_cache=true
116     become: true
117
118   - name: Install ML packages
119     become: true
120     pip:
121       name: tensorflow==2.10.0, keras==2.10.0, numpy<2.0, future

```

4.5 Questions

1. What is the difference between CloudInit and Ansible?

CloudInit runs once at first boot before SSH is available. It is meant for bootstrapping: creating users, adding SSH keys, and installing a small set of packages. It cannot be run again after the VM is created.

Ansible runs from a controller machine over SSH at any point in time. Playbooks are idempotent, meaning running them several times gives the same result. Ansible is built for configuration management and can apply changes to many machines at the same time.

In Task 3, I used CloudInit only for minimal setup. I then used Ansible to connect over SSH and handle all further configuration.

2. Explain the configurations available in dev-cloud-cfg.txt and prod-cloud-cfg.txt files

Both files share the same structure. Each one creates a single user named `appuser` with passwordless `sudo` access, a `/bin/bash` shell, and the cluster SSH public key added to `ssh_authorized_keys`. The `byobu_default: system` line activates the Byobu terminal multiplexer for interactive sessions. The files are deliberately minimal because Ansible handles all further configuration after the VM boots.

3. What problem have we solved by using Ansible?

Ansible solved the problem of configuring multiple machines at the same time. In Tasks 1 and 2, each VM had its own CloudInit file and was set up independently. With Ansible, one playbook command configured both the production server and the development server simultaneously. Common configuration steps are listed once under `hosts: all`. Server-specific steps go in their own section. If the configuration needs to change, the playbook is updated once and reapplied to all machines.

4. Summary

In this task, I extended the deployment from a single-server setup to a coordinated multi-server infrastructure. I provisioned two virtual machines through the OpenStack Python API from the client VM: the client itself, a production server, and a development server. I bootstrapped both servers with a minimal CloudInit file that created the `appuser` account with SSH access, which allowed Ansible to connect immediately after boot.

I then ran the Ansible playbook (`configuration.yml`) with a single command to configure both servers simultaneously. In the common play, I updated system packages and cloned the application repository on all hosts. In the production play, I installed Docker CE and started the application containers via `docker compose up -d`, replicating the Task 2 containerised deployment. In the development play, I installed Python, pip, and the ML packages needed to train new models with `neural_net.py`.

Compared to Tasks 1 and 2, the key improvement I gained is manageability. When I need to change the configuration, I edit a single playbook file and reapply it, rather than reprovisioning VMs with updated CloudInit files. The separation of roles I established—client for orchestration, production for serving, and development for training—also created the three-VM topology that the CI/CD pipeline in Task 4 builds directly on.

5 Task 4: CI/CD Using Git Hooks

5.1 Overview

Task 4 requires setting up a continuous deployment pipeline using Git Hooks on top of the multi-server infrastructure from Task 3. A bare Git repository must be created on the production server, and a `post-receive` hook must be configured to automatically deploy new model files to the live application folder whenever the development server pushes to the master branch. The goal is to eliminate any manual file copying or service restart from the model-update workflow.

5.2 Architecture

The pipeline uses the same three VMs from Task 3:

- **Development server:** trains a new model with `neural_net.py` and pushes the model files to the production server over Git.
- **Production server:** has a bare Git repository at `/home/appuser/my_project`. The `post-receive` hook copies new files to the live application folder on every push.
- **Client VM:** used in Task 3 to provision and configure the production and development servers via Ansible. It is not part of the ongoing push pipeline once the infrastructure is in place.

```

Dev Server --git push (SSH)--> Prod bare repo --post-receive hook-->
                               /model_serving/ci_cd/production_server

```

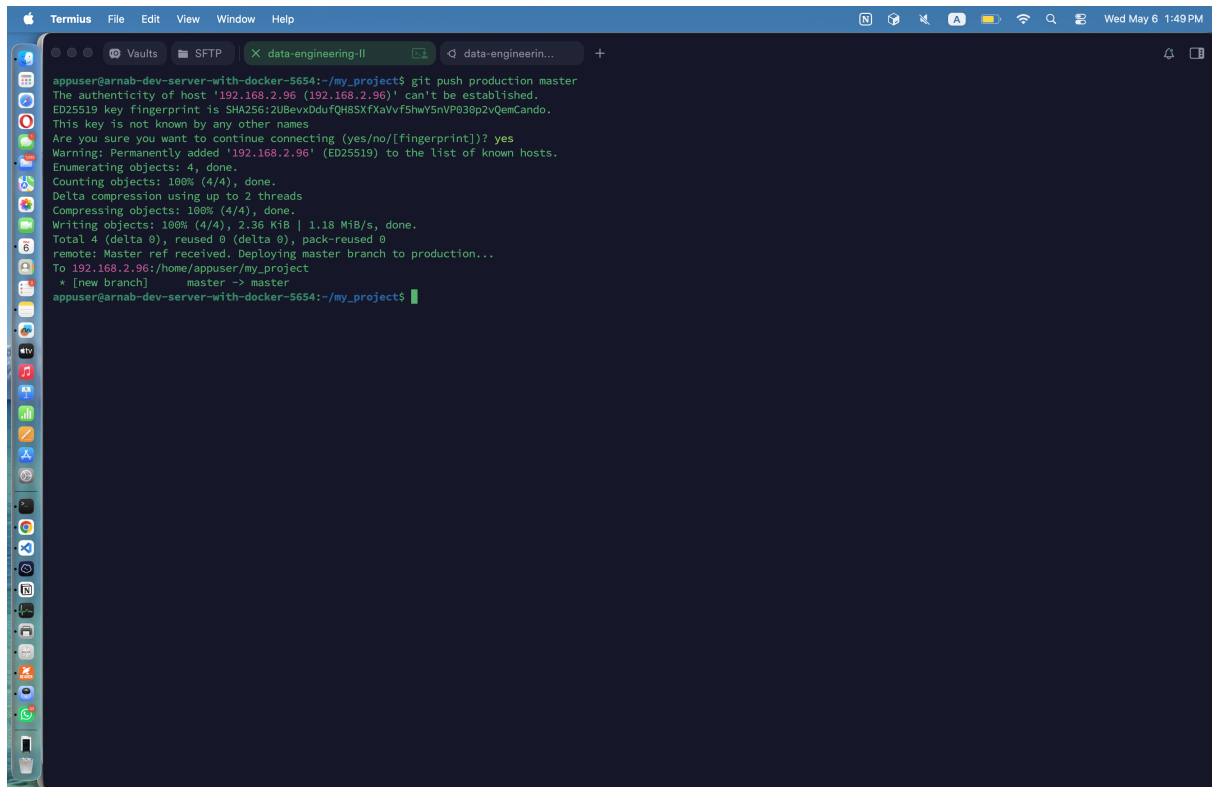



Figure 8: Deployment pipeline: development server pushes model files to production server via Git, triggering the post-receive hook for automated deployment.

5.3 The post-receive Hook

The hook file is at `/home/appuser/my_project/hooks/post-receive` on the production server:

Listing 8: post-receive Git hook on the production server

```
#!/bin/bash
while read oldrev newrev ref
do
  if [[ $ref =~ .*/master$ ]];
  then
    echo "Master ref received. Deploying master branch to production..."
    sudo git --work-tree=/model_serving/ci_cd/production_server \
      --git-dir=/home/appuser/my_project checkout -f
  else
    echo "Ref $ref successfully received. Doing nothing: only the master \
branch may be deployed on this server."
  fi
done
```

The hook reads each pushed branch name. If the branch is `master`, it runs `git checkout -f` with `-work-tree` to copy the pushed files into the live application folder. Old model files are replaced. If any other branch is pushed, the hook prints a message and does nothing.

5.4 Running the Pipeline

Listing 9: Initial push from the development server

```
# On the development server
mkdir /home/appuser/my_project && cd /home/appuser/my_project
```

```
git init
cp /model_serving/ci_cd/development_server/model* .
git add .
git commit -m "initial model"
git remote add production appuser@<PROD-IP>:/home/appuser/my_project
git push production master
```

Listing 10: Updating the model and pushing to production

```
# Train a new model on the development server
sudo python3 /model_serving/ci_cd/development_server/neural_net.py

# Copy new model files to the git working directory
cp /model_serving/ci_cd/development_server/model* /home/appuser/my_project/

# Commit and push
cd /home/appuser/my_project
git add .
git commit -m "updated model"
git push production master
```

The `neural_net.py` script trains a new model and saves updated model files. When pushed, the post-receive hook automatically deploys these new files to production. The expected output confirms the hook ran:

```
remote: Master ref received. Deploying master branch to production...
To 192.168.x.x:/home/appuser/my_project
    c20d19e..666ca0c  master -> master
```

5.5 Questions

1. What are Git hooks? Explain the post-receive script.

Git hooks are shell scripts that Git runs at certain points in the workflow. They are stored in the `hooks/` folder of a repository. Each hook has a fixed name that matches the event it listens to.

The post-receive script in this task runs on the production server after a push is fully received. It reads each pushed branch. If the branch is `master`, it runs:

```
sudo git --work-tree=/model_serving/ci_cd/production_server \
    --git-dir=/home/appuser/my_project checkout -f
```

This copies the files from the push into the live folder, replacing the current model files. If any other branch is pushed, the hook only prints a message and stops. This makes sure only master commits reach production.

2. Why is the bare Git repository needed on the production server?

A bare repository stores only the Git object database with no working tree. It serves as the remote that the development server pushes to. A regular repository cannot accept a push to its active branch because Git cannot safely update the working tree during the push. A bare repository has no active branch and no working files, so it always accepts pushes. After the push completes, the post-receive hook copies files to a separate live folder using `-work-tree`. This separation keeps Git storage and the running application files in different locations.

3. Four Git hooks and their functionalities.

1. **pre-commit**: runs before a commit is saved. It is often used to run linters or tests. If it exits with an error, the commit is cancelled.
2. **commit-msg**: runs after a commit message is written. It can check that the message follows a required format, such as including a ticket number.
3. **pre-receive**: runs on the server before any branch is updated. It can reject a push based on rules such as file size or branch naming.
4. **post-receive**: runs on the server after all branches are updated. It is used to trigger deployments, send alerts, or update build status—as shown in this task.

4. Kubernetes deployment diagram for the multi-server setup.

I could run the same application from Task 3 in Kubernetes. A **Deployment** manages the Flask web pods and exposes them via a **Service** of type **LoadBalancer**. RabbitMQ runs in a **StatefulSet** behind a headless **Service**. The Celery workers run in a separate **Deployment** that can be scaled independently. A **ConfigMap** stores non-secret settings like the RabbitMQ hostname. A **Secret** stores credentials. Both web pods and worker pods mount the same **PersistentVolumeClaim** to access model files.

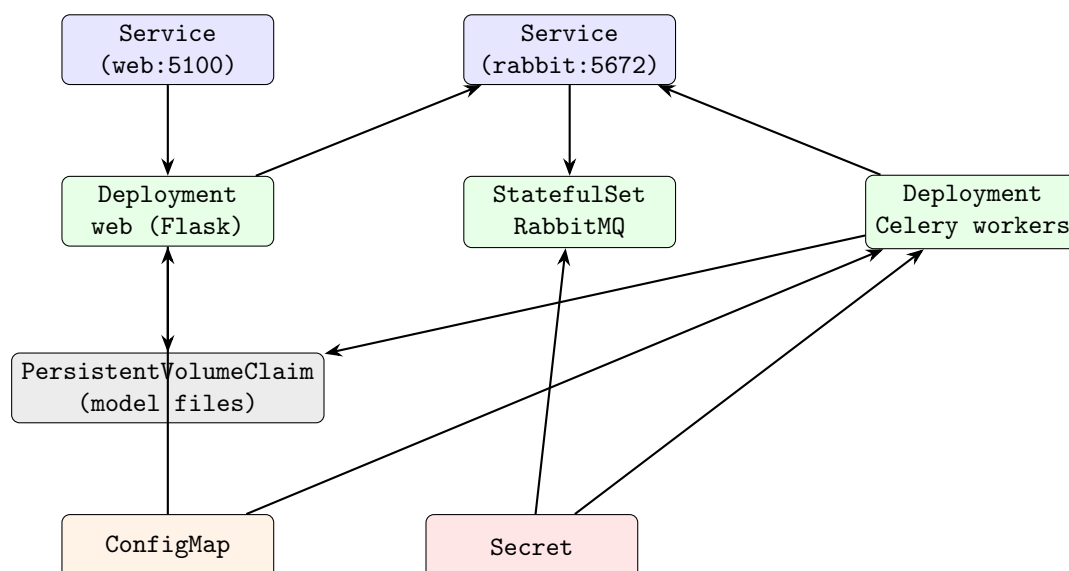


Figure 9: Kubernetes deployment diagram: Flask web Deployment behind a Service, RabbitMQ StatefulSet with headless Service, Celery worker Deployment, shared PersistentVolumeClaim for model files, ConfigMap and Secret for configuration and credentials.

5. Summary

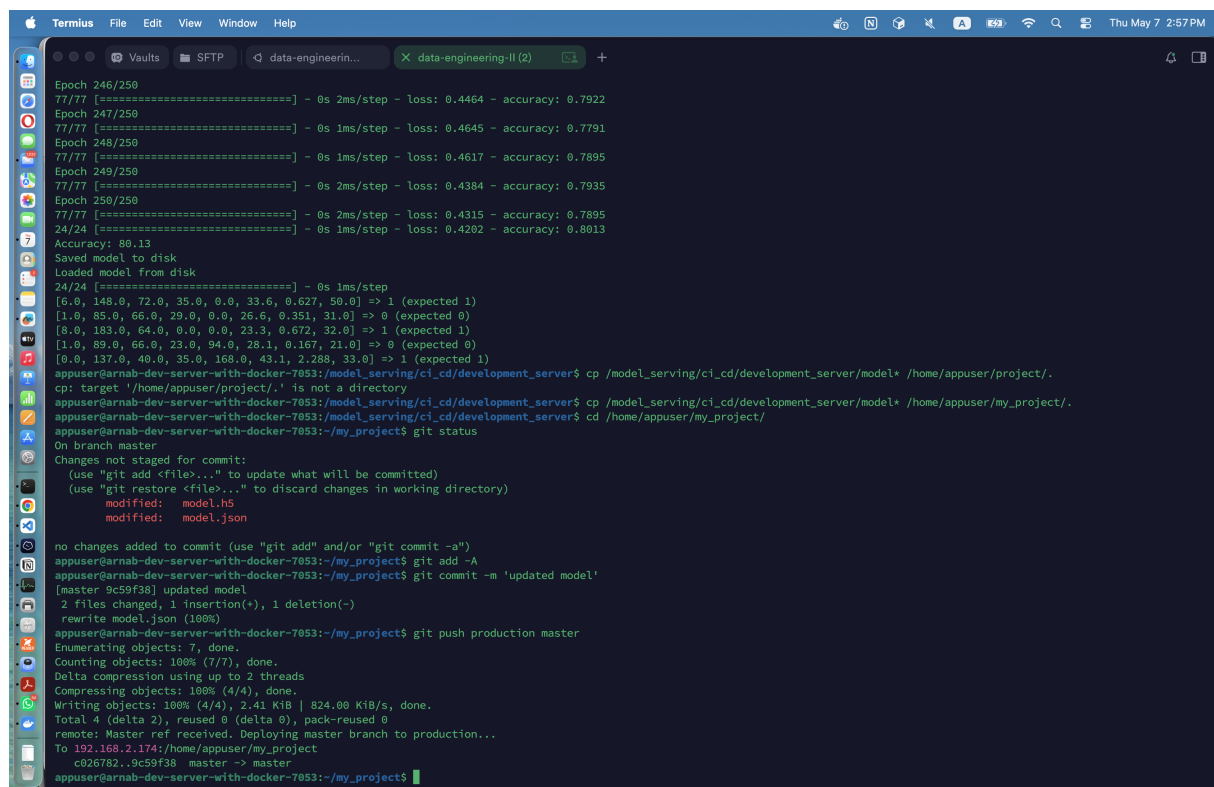
In this task, I completed the continuous deployment pipeline for the model serving application. I combined the multi-server infrastructure from Task 3 with a Git-based deployment mechanism. This created an end-to-end workflow where a retrained model is live in production within seconds of a `git push`, with no manual file copying or service restart needed.

I set up the pipeline using only Git and SSH—both of which were already present from Task 3. I created a bare repository on the production server as the deployment target and wrote the `post-receive` hook as the deployment agent. When I push new model files from the development server to the master branch, the hook is triggered immediately. It runs `git checkout -f`

with the `-work-tree` flag pointing to the live application folder. This replaces the old model files with the new ones. The entire deployment happens server-side with no further action needed from me.

I verified the pipeline end-to-end with a before-and-after test. Before the push, I confirmed the predictions page showed results from the original model. I then trained a new model by running `neural_net.py` on the development server, copied the updated files into the local Git working directory, committed, and pushed to the production bare repository. The hook deployed the new model files automatically, and I confirmed the predictions page reflected the updated model output, showing that the new `model.h5` and `model.json` were picked up by the running Celery worker.

I found this approach lightweight and effective for a single-server production setup. The main limitation I identified is that file replacement is not atomic: a worker reading the model during a push could briefly encounter an inconsistent state. In a production environment I would address this with a blue-green deployment or a container image swap, but for this task the simple file-copy approach demonstrated the core CI/CD concept clearly and proved reliable in practice.



```

Epoch 246/250
77/77 [=====] - 0s 2ms/step - loss: 0.4464 - accuracy: 0.7922
Epoch 247/250
77/77 [=====] - 0s 1ms/step - loss: 0.4645 - accuracy: 0.7791
Epoch 248/250
77/77 [=====] - 0s 1ms/step - loss: 0.4617 - accuracy: 0.7895
Epoch 249/250
77/77 [=====] - 0s 2ms/step - loss: 0.4384 - accuracy: 0.7935
Epoch 250/250
77/77 [=====] - 0s 2ms/step - loss: 0.4315 - accuracy: 0.7895
24/24 [=====] - 0s 1ms/step - loss: 0.4282 - accuracy: 0.8013
Accuracy: 80.13
Saved model to disk
Loaded model from disk
24/24 [=====] - 0s 1ms/step
[6.0, 148.0, 72.0, 35.0, 0.0, 33.6, 0.627, 50.0] => 1 (expected 1)
[1.0, 85.0, 66.0, 29.0, 0.0, 26.6, 0.351, 31.0] => 0 (expected 0)
[8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0] => 1 (expected 1)
[1.0, 89.0, 66.0, 23.0, 94.0, 28.1, 0.167, 21.0] => 0 (expected 0)
[0.0, 137.0, 40.0, 35.0, 168.0, 43.1, 2.288, 33.0] => 1 (expected 1)
appuser@arnab-dev-server-with-docker-7053:/model_serving/ci_cd/development_server$ cp /model_serving/ci_cd/development_server/model* /home/appuser/project/
cp: target '/home/appuser/project/' is not a directory
appuser@arnab-dev-server-with-docker-7053:/model_serving/ci_cd/development_server$ cp /model_serving/ci_cd/development_server/model* /home/appuser/my_project/
appuser@arnab-dev-server-with-docker-7053:/model_serving/ci_cd/development_server$ cd /home/appuser/my_project/
appuser@arnab-dev-server-with-docker-7053:~/my_project$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   model.h5
        modified:   model.json
no changes added to commit (use "git add" and/or "git commit -a")
appuser@arnab-dev-server-with-docker-7053:~/my_project$ git add -A
appuser@arnab-dev-server-with-docker-7053:~/my_project$ git commit -m 'updated model'
[master 9c59f38] updated model
 2 files changed, 1 insertion(+), 1 deletion(-)
rewrite model.json (100%)
appuser@arnab-dev-server-with-docker-7053:~/my_project$ git push production master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.41 KiB | 824.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Master ref received. Deploying master branch to production...
To 192.168.2.174:/home/appuser/my_project
 c026782..9c59f38  master -> master
appuser@arnab-dev-server-with-docker-7053:~/my_project$

```

Figure 10: Terminal output of `git push production master` confirming that the post-receive hook ran and deployed the updated model files to the live application folder on the production server.

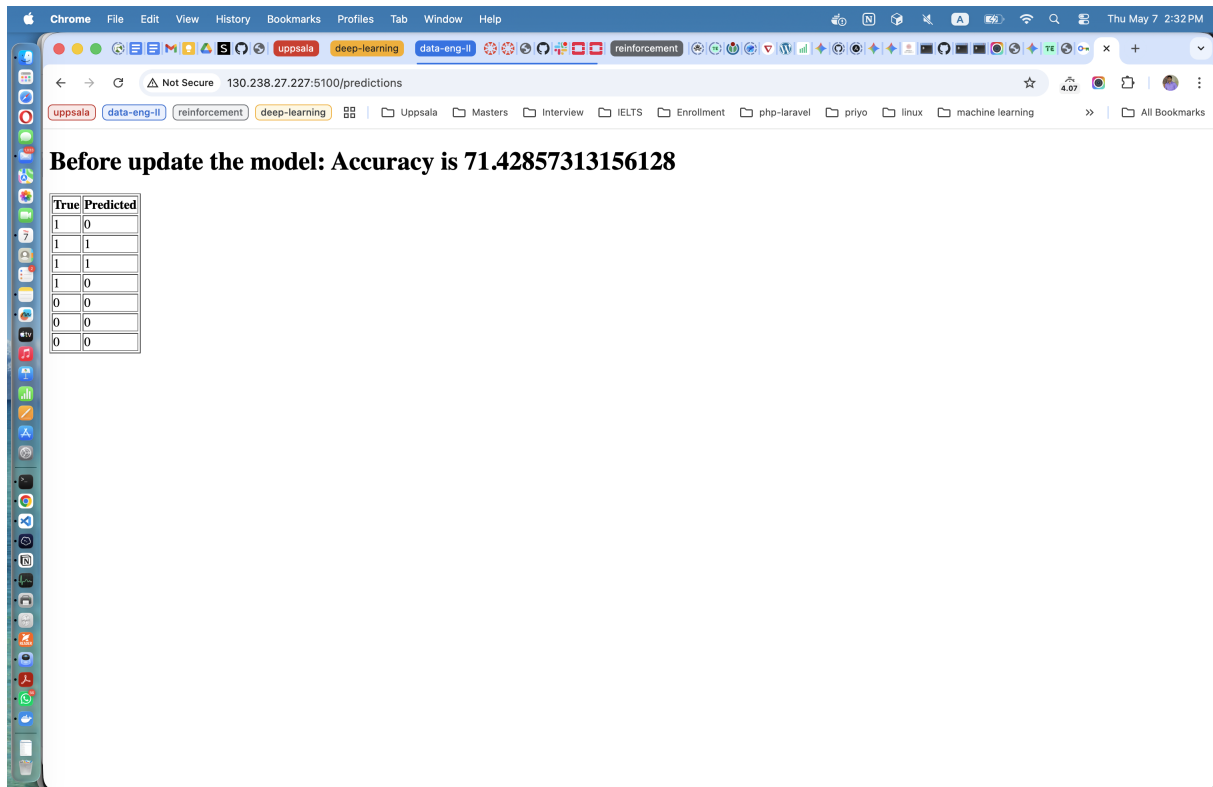


Figure 11: Predictions page before the CI/CD pipeline push, showing inference results from the original trained model.

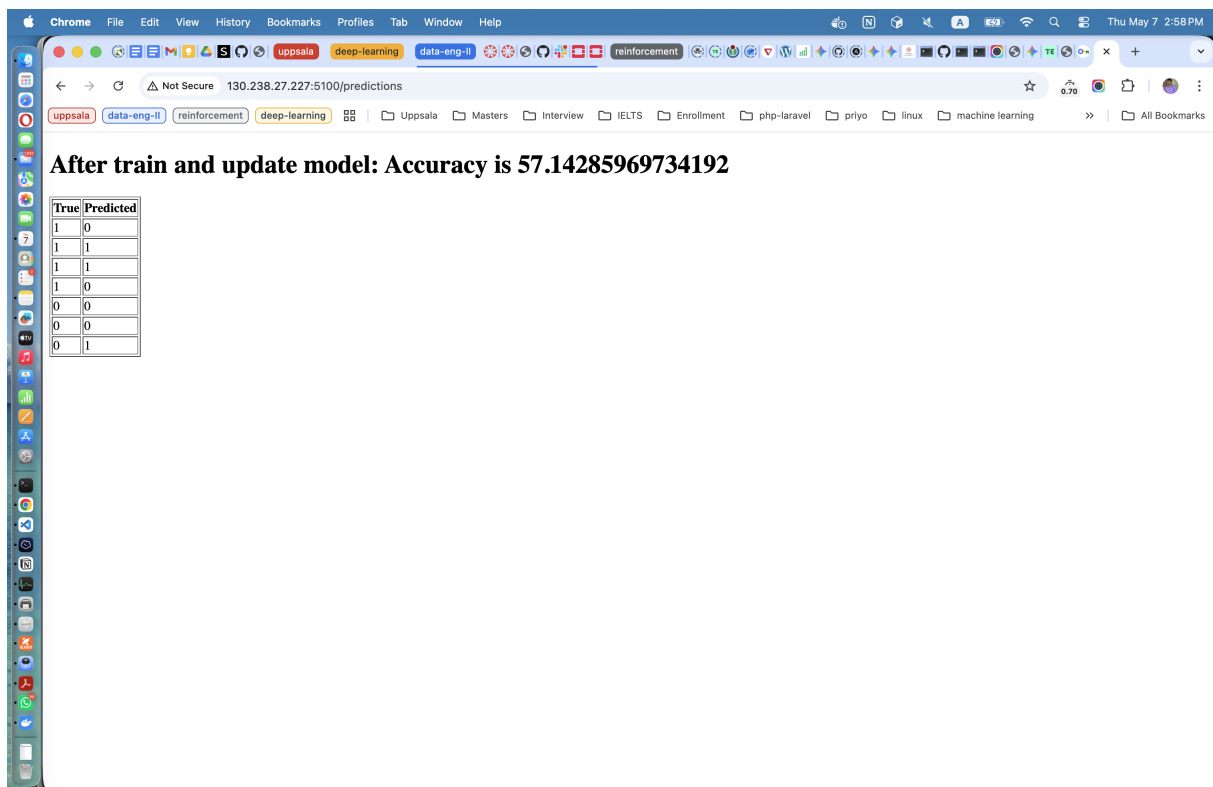


Figure 12: Predictions page after the CI/CD push, showing updated inference results from the retrained model deployed automatically via the post-receive hook.

6 Conclusion

This assignment demonstrated a progressive path toward reliable cloud deployment and continuous deployment. Task 1 established basic manual deployment with CloudInit. Task 2 improved this with Docker containers, providing isolation and easy scaling. Task 3 scaled further with Ansible, managing multiple servers from a single control point. Task 4 completed the cycle by automating model updates through a Git-based CI/CD pipeline.

Together, these tasks show how modern infrastructure tools address real deployment challenges: reproducibility, scalability, configuration management, and automation. The progression from single-server CloudInit deployment to multi-server Ansible orchestration with automated CI/CD demonstrates industry best practices for production infrastructure.